

SIGA-INEBI

Manual de Instalación y Configuración

Cómo preparar, desplegar, configurar y verificar operativamente la plataforma en entornos con Docker y ambientes locales nativos

Ficha del documento	
Sistema	SIGA-INEBI — plataforma web académica, administrativa y operativa
Audiencia	Desarrolladores, revisores técnicos, administradores de infraestructura y evaluadores
Base documental	**AGENTS.md**, **docs/architecture/**, **docs/decisions/**, **docs/manuals/03-installation-and-configuration-instructions.md**, Docker y scripts
Estado del código analizado	Rama develop, validado en Docker Compose y entorno local nativo (Windows/Linux)
Fecha de elaboración	4 de octubre de 2026
Sustituye a	Guías técnicas y borradores no unificados de puesta en marcha

Cómo leer este manual

El manual sigue el orden pedido para el entregable: alcance y arquitectura, requisitos previos y dimensionamiento, gestión de variables de entorno (.env), despliegue con Docker Compose, despliegue local nativo (sin Docker), verificación funcional y pruebas, y diagnóstico y resolución de problemas. Se añade un capítulo final de contingencias reales verificadas y dos anexos técnicos con la matriz completa de comandos y la trazabilidad de requerimientos no funcionales.

Convenciones

Marca	Significado
`código`	Nombre de archivo, comando, campo, endpoint o variable, tal como aparece en el repositorio.
No implementado	La capacidad se menciona en documentos o requisitos pero no existe en el código. Se indica su referencia.
Recuadro azul	Nota informativa o directriz técnica de la arquitectura de la plataforma.
Recuadro ámbar	Atención: advertencias de entorno, diferencias entre sistemas operativos o limitaciones operativas.
Recuadro rojo	Capacidad no implementada, peligro operativo o riesgo de pérdida de datos.
`[lee] · [escribe] · [destruye]`	En los comandos: efecto sobre datos y archivos persistentes.

Contenido

Manual de Instalación y Configuración • SIGA-INEBI

Capítulo	Temas que incluye	Pág.
1	**Alcance, Objetivos y Modelo Arquitectónico:** Propósito, límites, monorrepositorio, monolito modular, servicios y topología local.	04
2	**Requisitos Previos y Dimensionamiento:** Hardware mínimo y recomendado (RNF-CAP-001), versiones canónicas de software, matriz de puertos.	05
3	**Gestión de Configuración (.env):** Plantillas oficiales (.env.example), variables por grupo funcional, perfiles (dev, test, prod) y PostgreSQL.	06
4	**Despliegue con Docker Compose (Recomendado):** Arquitectura del stack, contenedores db, backend y frontend, volúmenes nombrados y Makefile.	07
5	**Despliegue Local Nativo (Sin Docker):** Entorno virtual Python, dependencias, migraciones, script dev.mjs en Vite y siembra institucional.	08
6	**Verificación Funcional y Acceso Inicial:** Endpoints HTTP de sanidad, cuentas de demostración (admin / 1234), suites de prueba pytest y vitest.	10
7	**Diagnóstico y Resolución de Problemas:** Conflictos de puertos, bloqueos de OneDrive, intérprete global vs venv y reintentos de conexión PostgreSQL.	11
Anexos	**Anexos Técnicos de Referencia (A y B):** Matriz completa de comandos del monorrepositorio y trazabilidad de requerimientos no funcionales (RNF).	12

1. Alcance, Objetivos y Modelo Arquitectónico

1.1 Propósito del Documento

El objetivo fundamental de este manual es definir el procedimiento técnico oficial para la preparación, configuración, ejecución y validación del sistema SIGA-INEBI. Toda persona técnica que reciba una copia limpia del repositorio debe ser capaz de levantar el ambiente completo de forma predecible, determinista y auditable.

1.2 Audiencia Objetivo

El documento está dirigido a desarrolladores de software, ingenieros de control de calidad (QA), personal técnico del Ministerio de Educación (MINEDUC) y administradores de sistemas responsables del mantenimiento de los ambientes del Instituto Nacional de Educación Básica de Salcajá.

1.3 Organización del Monorrepositorio (ADR-0001)

Conforme a la decisión de arquitectura ADR-0001, todo el sistema reside en un único repositorio. Esta estructura garantiza que cualquier cambio a los contratos de la API REST, los componentes de interfaz, las migraciones de base de datos o los documentos de trazabilidad se revisen y versionen en conjunto:

```
siga-inebi/
|-- backend/               # Monolito modular Django 5.2 + Django REST Framework
|   |-- apps/              # 12 aplicaciones desacopladas por dominio de negocio
|   |-- config/            # Configuracion de Django, URLs y settings por entorno
|   |-- requirements/      # Dependencias congeladas (base.txt, dev.txt, test.txt)
|   \-- manage.py          # Punto de entrada de la CLI administrativa de Django
|-- frontend/              # Aplicacion de pagina unica (SPA) en React 19 + Vite 7
|   |-- src/               # Componentes agrupados por dominio institucional y vistas
|   |-- scripts/           # Scripts de ejecucion segura (dev.mjs) y control de presupuesto
|   \-- package.json       # Definicion de dependencias y scripts de Node.js
|-- docs/                  # Documentacion de arquitectura, manuales y trazabilidad
|-- scripts/               # Utilidades de respaldo, provision local y gobierno
|-- compose.yml            # Declaracion base de servicios para Docker Compose
|-- compose.override.yml   # Ajuste para desarrollo local (montajes y puertos)
\-- Makefile               # Atajos de orquestacion estandarizados para ingenieria
```

1.4 Arquitectura del Monolito Modular (ADR-0002 y ADR-0008)

El backend se organiza como un monolito modular. No existen microservicios independientes; el servicio corre en un único proceso HTTP optimizado. Internamente, cada aplicación en backend/apps/ respeta una estricta separación en capas que aísla las reglas de negocio de los serializadores y vistas web:

- **Capa API (api/views.py, api/serializers.py):** Recibe peticiones HTTP, valida formatos JSON de entrada, aplica autenticación y permisos atómicos, delegando la lógica de negocio.
  - **Capa de Consulta (queries.py):** Consultas de lectura optimizadas mediante el ORM de Django. No conoce objetos Request ni serializadores.
  - **Capa de Servicios y Dominio (services.py):** Ejecuta las operaciones de escritura, transacciones atómicas, invariantes de negocio y emite eventos de auditoría (apps.audit).
  - **Capa de Modelos (models.py):** Declaración física de entidades, llaves foráneas y restricciones.

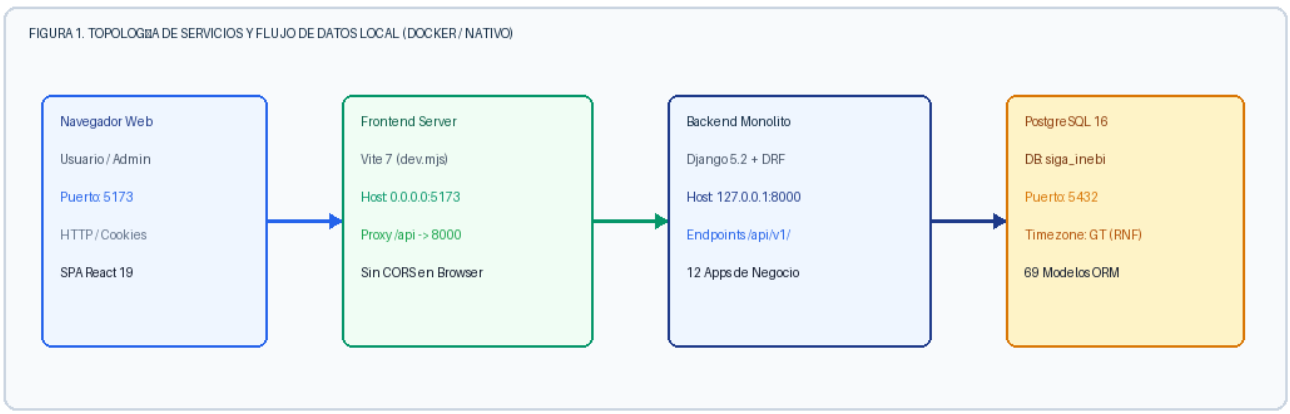


Figura 1. *Topología de servicios, canalización de proxy y flujo de datos local (Docker / Nativo).*

2. Requisitos Previos y Dimensionamiento

Antes de iniciar la instalación de SIGA-INEBI en cualquier máquina o servidor, se debe verificar el cumplimiento de los estándares de hardware, software y red aquí indicados.

2.1 Dimensionamiento de Hardware Mínimo

El dimensionamiento responde al requisito no funcional RNF-CAP-001, diseñado para garantizar operación fluida en hardware institucional de bajo costo:

Recurso	Requisito Mínimo (RNF-CAP-001)	Recomendado (Desarrollo)	Justificación Técnica
CPU	1 vCPU (x86_64 o arm64)	2 núcleos físicos o más	Procesamiento web no intensivo; el monolito optimiza I/O.
Memoria RAM	2.0 GB dedicados	4.0 GB a 8.0 GB	Soporta PostgreSQL 16, Django y servidor de estáticos.
Almacenamiento	10.0 GB en disco SSD	20.0 GB libres	RNF-CAP-002: estimación de crecimiento de 2 GiB por ciclo.
Red	10 Mbps / Conexión local	100 Mbps o superior	Tráfico liviano de APIs JSON y credenciales QR.

2.2 Versiones de Software y Herramientas Base

Componente	Versión Verificada	Finalidad en SIGA-INEBI	Comando de Validación
Git	>= 2.40.0	Control de versiones y gestión del monorrepositorio.	`git --version`
Docker Engine	>= 24.0.0	Aislamiento de servicios y paridad de entornos.	`docker --version`
Docker Compose	>= 2.20.0 (v2)	Orquestación multinodo (db, backend, frontend).	`docker compose version`
Python (Modo Nativo)	3.12.x LTS (o 3.11)	Ejecución del monolito Django sin contenedores.	`python --version`
Node.js (Modo Nativo)	>= 20.18.0 (Node 22 ok)	Empaquetado y servidor de desarrollo de React 19.	`node --version`
PostgreSQL (Modo Nativo)	16.x LTS	Motor relacional ACID institucional obligatorio.	`psql --version`

2.3 Matriz de Puertos y Protocolos

Puerto	Servicio	Protocolo	Propósito y Restricción de Exposición
5173	Frontend Vite	HTTP / WebSocket HMR	Interfaz web SPA; expuesto en desarrollo local (0.0.0.0:5173).
8000	Backend Django API	HTTP (REST JSON)	API REST y panel administrativo; consumido vía proxy o directo.
5432	PostgreSQL	TCP / IP (Postgres)	Base de datos; en Compose solo debe exponerse si se requiere inspección externa.

Atención: Conflictos de Puertos

Verificación de Puertos Libres: Antes de iniciar los contenedores o procesos nativos, asegúrese de que los puertos 5173, 8000 y 5432 no estén tomados por otros servicios como IIS, servicios locales de Postgres preinstalados o instancias huérfanas de Node. En Windows PowerShell: Get-NetTCPConnection -LocalPort 5173,8000,5432 -ErrorAction SilentlyContinue

3. Gestión de Configuración y Variables (.env)

Toda la configuración del sistema se gobierna mediante variables de entorno declaradas en el archivo raíz .env y sus complementos modulares. El repositorio provee plantillas oficiales denominadas .env.example, backend/.env.example y frontend/.env.example.

3.1 Preparación de Archivos de Entorno

Para inicializar un entorno limpio, copie las plantillas a sus destinos activos. Nunca altere los archivos .example con credenciales reales ni suba los archivos .env a Git:

```
# En Linux / macOS / WSL:
cp .env.example .env && cp backend/.env.example backend/.env && cp frontend/.env.example frontend/.env
# En Windows (PowerShell):
Copy-Item .env.example .env; Copy-Item backend/.env.example backend/.env; Copy-Item frontend/.env.example frontend/.env
```

3.2 Grupos de Variables por Finalidad Operativa

Variable de Entorno	Valor Predeterminado (Dev)	Propósito y Efecto en la Pila
DJANGO_SETTINGS_MODULE	config.settings.development	Selecciona el perfil de ejecución (development, testing, production).
SECRET_KEY	*(cadena aleatoria dev)*	Firma criptográfica de sesiones y tokens. En producción debe ser de 50+ caracteres aleatorios.
DEBUG	True	Muestra trazas detalladas de error. Debe ser obligatoriamente False en producción.
ALLOWED_HOSTS	localhost,127.0.0.1,backend	Lista de nombres de host HTTP autorizados para atender solicitudes.
DB_HOST	db (Docker) / 127.0.0.1	Nombre de red o IP del motor PostgreSQL.
DB_PORT	5432	Puerto TCP de escucha de la base de datos relacional.
DB_NAME	siga_inebi	Nombre de la base de datos PostgreSQL del establecimiento.
DB_USER / DB_PASSWORD	siga_user / siga_secret_dev	Credenciales técnicas de conexión al gestor de base de datos.
CORS_ALLOWED_ORIGINS	http://localhost:5173	Orígenes de navegador autorizados para realizar peticiones de API con credenciales.
TIME_ZONE	America/Guatemala	Cumplimiento estricto del RNF-LOC-001 para marcas de tiempo y auditoría.

3.3 Diferenciación de Perfiles de Ejecución

El sistema distingue tres perfiles canónicos gobernados por DJANGO_SETTINGS_MODULE:
• **Desarrollo (config.settings.development):** Activa recarga en caliente (autoreload), permite orígenes locales en CORS y tolera encabezados no seguros de depuración.
• **Pruebas Automatizadas (config.settings.testing):** Optimiza la velocidad de ejecución de pruebas con contraseñas rápidas (MD5 hasher), desactiva trazas ruidosas y aísla la base de datos de test.
• **Producción (config.settings.production):** Exige DEBUG=False, fuerza cookies con atributos Secure, HttpOnly y SameSite=Lax, y rechaza arranques con secretos débiles.

Peligro: Prohibición de SQLite en Entregables

Política de Base de Datos (AGENTS.md Regla 7): El código del monorrepositorio exige PostgreSQL para todos los entornos de prueba, integración y despliegue. No está permitido recurrir a SQLite para saltarse dependencias en entregables oficiales, ya que oculta incompatibilidades de tipos de datos JSONB, búsquedas de texto y bloqueos transaccionales.

4. Despliegue con Docker Compose (Recomendado)

La forma estándar y recomendada por la arquitectura institucional para ejecutar SIGA-INEBI es mediante Docker Compose. Este mecanismo asegura aislamiento completo de dependencias, garantiza paridad de versiones entre desarrolladores y emula fielmente el entorno productivo.

4.1 Servicios que Componen el Stack Local

Servicio	Imagen / Build	Puerto Host	Volumen / Persistencia	Responsabilidad Operativa
`db`	postgres:16-alpine	5432:5432	`postgres_data`	Motor de datos relacional; inicializa base con variables del .env.
`backend`	./backend/Dockerfile	8000:8000	`./backend:/app`	API Django REST Framework, migraciones y utilidades de gestión.
`frontend`	./frontend/Dockerfile	5173:5173	`./frontend:/app`	Servidor de desarrollo Vite con Hot Module Replacement (HMR).

4.2 Secuencia de Arranque Paso a Paso

Abra una terminal en la raíz del monorrepositorio y ejecute la siguiente secuencia canónica:

```
# 1. Preparar variables y levantar servicios en segundo plano:
cp .env.example .env && docker compose up -d --build
# 2. Monitorear logs hasta inicialización de la base: docker compose logs -f
# 3. Aplicar migraciones del ORM y siembra institucional [escribe]:
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py seed_demo_data
# 4. Comprobar salud de los contenedores: docker compose ps
```

4.3 Volúmenes Nombrados y Persistencia de Datos

El archivo compose.yml declara el volumen persistente postgres_data. Esto asegura que detener los contenedores con docker compose down **NO destruya** los datos académicos, notas ni registros de auditoría.

> **Advertencia de Eliminación de Datos**
>
> **Operación Destructiva:** Ejecutar docker compose down -v elimina los volúmenes nombrados, borrando por completo la base de datos postgres_data. Úselo únicamente si desea reiniciar el entorno desde un estado limpio absoluto.

4.4 Uso de Atajos con Makefile

El repositorio cuenta con un Makefile oficial que estandariza las operaciones comunes de ingeniería:

Target Make	Comando Equivalente	Efecto Operativo
`make up`	`docker compose up -d`	Levanta los contenedores en segundo plano.
`make down`	`docker compose down`	Detiene y remueve los contenedores conservando datos.
`make migrate`	`docker compose exec backend python manage.py migrate`	Aplica migraciones pendientes del ORM.
`make test`	`docker compose exec backend pytest && npm --prefix frontend test`	Corre las suites completas de pruebas unitarias.
`make logs`	`docker compose logs -f`	Muestra trazas de salida de todos los servicios en vivo.

5. Despliegue Local Nativo (Sin Docker)

En situaciones donde no se disponga de Docker Desktop o se requiera desarrollo y depuración directa, el monorrepositorio soporta ejecución nativa ejecutando Python y Node.js en el sistema operativo anfitrión (ADR-0007). A continuación se detalla el procedimiento validado para Windows (PowerShell) y Linux/macOS.

5.1 Requisitos de Base de Datos para Modo Nativo

Debe contarse con una instancia de PostgreSQL 16 ejecutándose en el puerto 5432. Cree la base y el usuario:

```
-- Conéctese a psql con el superusuario postgres:
CREATE USER siga_user WITH PASSWORD 'siga_secret_dev';
CREATE DATABASE siga_inebi OWNER siga_user;
GRANT ALL PRIVILEGES ON DATABASE siga_inebi TO siga_user;
```

5.2 Configuración y Arranque del Backend (Django)

Abra una terminal de consola en la carpeta raíz del proyecto y siga estos pasos:

```
# 1. Navegar a la carpeta backend
cd backend

# 2. Crear el entorno virtual aislado (.venv)
# En Linux/macOS:
python3 -m venv .venv
source .venv/bin/activate

# En Windows (PowerShell):
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 3. Actualizar pip e instalar dependencias del monolito
python -m pip install --upgrade pip
pip install -r requirements/dev.txt

# 4. Configurar variables locales del backend
Copy-Item .env.example .env    # Windows PowerShell
# cp .env.example .env         # Linux / macOS

# 5. Aplicar migraciones del ORM [escribe]
python manage.py migrate

# 6. Ejecutar la siembra institucional y superadministrador [escribe]
python manage.py seed_demo_data

# 7. Iniciar el servidor de desarrollo en localhost:8000
python manage.py runserver 127.0.0.1:8000
```

> **Atención: Resolución del Intérprete en Windows**
>
> **Causa Frecuente de Error en Windows:** Si se ejecuta simplemente python manage.py runserver sin activar el entorno virtual o si la política de ejecución de PowerShell bloquea Activate.ps1, Windows utilizará el intérprete global de Python (ej. C:\Python311), generando errores de ModuleNotFoundError: No module named 'drf_spectacular'.
> Solución inmediata: Invoque siempre el binario explícito del venv: .\.venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000

5.3 Configuración y Arranque del Frontend (React + Vite)

El frontend es una aplicación moderna construida sobre React 19 y empaquetada con Vite 7. Para correrlo localmente sin Docker, abra una segunda terminal independiente:

```
# 1. Navegar a la carpeta del frontend
cd frontend

# 2. Configurar variables de entorno del frontend
Copy-Item .env.example .env    # Windows PowerShell
# cp .env.example .env         # Linux / macOS

# 3. Instalar paquetes de npm (utilizar Node 20 LTS o 22)
npm install

# 4. Iniciar el servidor de desarrollo
npm run dev
```

> **Aviso Técnico: Manejo Seguro de Vite en Windows**
>
> **Mitigación Windows / OneDrive (dev.mjs):** En terminales no interactivas o cuando el proyecto reside en carpetas sincronizadas por Microsoft OneDrive, el proceso interactivo de Vite puede bloquearse por lectura de stdin. El repositorio cuenta con el script optimizado frontend/scripts/dev.mjs que ignora la entrada estándar y arranca el servidor HTTP limpiamente. Puede invocarlo directamente con node scripts/dev.mjs.

5.4 Siembra Inicial de Datos de Demostración

El comando seed_demo_data es idempotente y aprovisiona el catálogo maestro mínimo requerido para operar la plataforma en cualquier ambiente:

Entidad Aprovisionada	Cantidad / Identificador	Detalle Funcional y Propósito en Pruebas
Institución Educativa	INEBI Salcajá (INEBI-001)	Establecimiento oficial en Quetzaltenango con jornadas Matutina y Vespertina.
Ciclo Escolar	Ciclo 2026 (Abierto)	Periodo lectivo activo para registro de inscripciones, notas y asistencia.
Grados y Secciones	1.º, 2.º y 3.º Básico (A y B)	6 secciones con cupos y asignaciones docentes predefinidas.
Materias (CNB)	10 asignaturas oficiales	Currículo Nacional Base: Matemáticas, Comunicación, Ciencias Naturales, etc.
Roles RBAC Institucionales	6 roles oficiales	system-administrator, director, academic-coordinator, teacher, secretary, etc.
Superadministrador Demo	`admin / 1234`	Usuario institucional con permisos globales para pruebas funcionales inmediatas.
Estudiantes	100 estudiantes	85 matriculados en secciones y 15 disponibles para pruebas de matrícula.

6. Verificación Funcional y Acceso Inicial

Una vez iniciados los servicios de backend y frontend (sea en Docker o en modo nativo), se debe realizar la validación de sanidad operativa mediante los siguientes puntos de verificación.

6.1 URLs y Endpoints de Sanidad Técnica

Servicio	URL Local	Código Esperado	Respuesta de Sanidad
Frontend SPA	`http://localhost:5173/`	200 OK	HTML con formulario de inicio de sesión.
Backend Health	`http://127.0.0.1:8000/api/health/`	200 OK	`{"status": "healthy"}`
Documentación OpenAPI	`http://127.0.0.1:8000/api/docs/`	200 OK	Interfaz Swagger UI con esquemas de API.
Esquema OpenAPI	`http://127.0.0.1:8000/api/schema/`	200 OK	Documento YAML / JSON de contratos públicos.

6.2 Cuentas Institucionales de Demostración

El seeder crea usuarios representativos de cada estamento escolar para validar la denegación por defecto (RBAC):

Rol Institucional	Usuario / Correo	Contraseña y Alcance Funcional
Superadministrador	`admin`	1234 — Acceso absoluto, configuración global y auditoría.
Director General	`director.demo@siga.test`	1234 — Gestión institucional, consulta de matrícula y notas.
Secretaría Académica	`secretaria.demo@siga.test`	1234 — Inscripción de alumnos y expedientes académicos.
Docente Titular	`docente.demo@siga.test`	1234 — Ingreso de notas y control de asistencia de sus cursos.

6.3 Ejecución de Pruebas Automatizadas

Para validar que la instalación no presenta regresiones, corra las suites de verificación:

```
# Pruebas de Backend (pytest con soporte de cobertura):
cd backend
python -m pytest

# Pruebas de Frontend (Vitest con suite de navegación y autenticación):
cd ../frontend
npm test -- --run
```

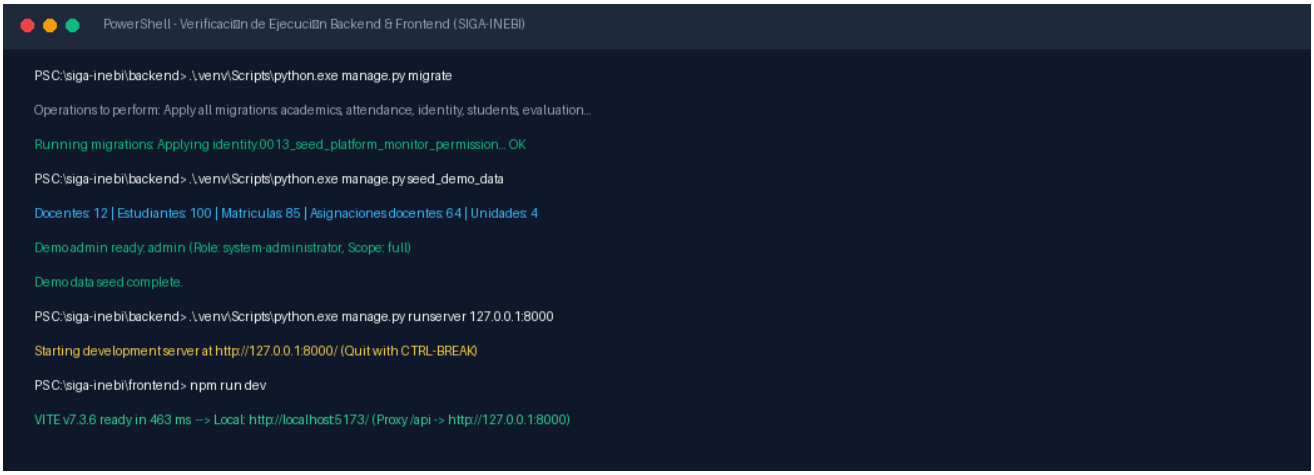


Figura 2. *Verificación de ejecución de migraciones, siembra institucional y arranque de servidores en terminal.*

7. Diagnóstico y Resolución de Problemas

En esta sección se consolidan las anomalías técnicas más frecuentes observadas durante la puesta en marcha en diferentes sistemas operativos y sus soluciones inmediatas comprobadas.

Síntoma / Mensaje de Error	Causa Raíz Identificada	Solución Técnica Paso a Paso
`Error: listen EADDRINUSE: address already in use :::5173`	Instancia previa de Vite o Node ocupando el puerto 5173.	Liberar puerto: Windows: Stop-Process -Id (Get-NetTCPConnection -LocalPort 5173).OwningProcess -Force Linux: fuser -k 5173/tcp
`ModuleNotFoundError: No module named 'drf_spectacular'`	Ejecución bajo el intérprete global de Windows en lugar del entorno virtual .venv.	Activar el venv antes de correr o invocar directamente: .\.venv\Scripts\python.exe manage.py runserver
`EBUSY: resource locked o ENOENT en Windows OneDrive`	El motor de sincronización de OneDrive bloquea archivos temporales creados por npm o vite.	1. Pausar la sincronización de OneDrive durante el desarrollo. 2. Utilizar el ejecutor seguro node scripts/dev.mjs.
`django.db.utils.OperationalError: could not connect to server`	PostgreSQL no está iniciado o Docker Compose no ha terminado de montar el socket.	1. Verificar estado de Postgres: docker compose ps db. 2. Comprobar que DB_HOST sea db en Docker y 127.0.0.1 en nativo.
`Pantalla en blanco en navegador al cargar http://localhost:5173`	Error de JavaScript en el bundle o API de backend inaccesible.	1. Abrir la consola de DevTools (F12) y revisar la pestaña Red. 2. Verificar que http://127.0.0.1:8000/api/health/ responda 200.

Anexos Técnicos de Referencia

Anexo A: Matriz Completa de Comandos del Monorrepositorio

Operación Técnica	Comando Canónico (CLI)	Contexto de Ejecución
Levantar Pila Docker	`docker compose up -d --build`	Raíz del repositorio; modo recomendado.
Detener Pila Docker	`docker compose down`	Conserva volúmenes de base de datos.
Aplicar Migraciones	`python manage.py migrate`	Dentro de backend/ con .venv activo.
Ejecutar Seeder Demo	`python manage.py seed_demo_data`	Genera catálogo y superadministrador.
Iniciar Backend Nativo	`python manage.py runserver 127.0.0.1:8000`	Escucha en puerto 8000.
Iniciar Frontend Nativo	`npm run dev`	Dentro de frontend/ con Node 22.
Ejecutar Pruebas Backend	`pytest --cov=apps`	Genera reporte de cobertura de código.
Validar Tipografía y Estilo	`npm run lint && ruff check .`	Control de calidad estática frontend y backend.

Anexo B: Trazabilidad con Requerimientos No Funcionales (RNF)

Código RNF	Requisito Oficial	Implementación Verificada en este Manual
`RNF-CAP-001`	Capacidad en hardware mínimo (1 vCPU, 2 GB RAM).	Verificado en Cap. 2.1; el stack optimiza memoria y reusa conexiones TCP con DATABASE_CONN_MAX_AGE.
`RNF-LOC-001`	Zona horaria oficial del establecimiento.	Fijada en America/Guatemala en .env y propagada a PostgreSQL y Django.
`RNF-LOC-002`	Monolingüe español de Guatemala.	Configurado es-gt en LANGUAGES en settings/base.py.
`RNF-OPE-001`	Nivel de registro operativo configurable.	Gobernado por DJANGO_LOG_LEVEL y TASK_LOG_LEVEL sin tocar código fuente.
`RNF-RES-001`	Respaldo y recuperación institucional.	Scripts en scripts/backup/ con RPO de referencia de 24h y RTO de 4h (PD-002).